Unity Catalog Governance

> **Document Owner:** Platform Team | **Status:** Approved | **Last Updated:** February 2026

Overview

Unity Catalog provides centralized governance for data and AI assets. This document defines the mandatory patterns for identity management, privilege assignment, and storage configuration.

Golden Rules

ID	Rule	Severity	WAF Pillar
GOV-01	Provision identities at account level via SCIM	Critical	Governance
GOV-02	Define and manage groups in your identity provider	Critical	Governance
GOV-03	Assign admin roles sparingly; avoid ALL PRIVILEGES	Required	Security
GOV-04	Use catalog-level managed storage for data isolation	Required	Security
GOV-05	Prefer managed tables over external tables	Required	Governance
GOV-06	Use service principals for production jobs	Required	Security
GOV-07	Assign object ownership to groups, not individuals	Required	Governance
GOV-08	Use lineage for impact analysis before schema changes	Required	Governance
GOV-09	Implement governance-as-code via system tables and automation	Required	Governance
GOV-10	Use row filters and column masks for fine-grained access control	Required	Governance
GOV-11	Configure audit logging for all workspaces	Required	Governance
GOV-12	Use ABAC policies for scalable tag-driven access control	Critical	Governance
GOV-13	Define ABAC policies at catalog level for maximum inheritance	Required	Governance
GOV-14	Use RBAC (privilege grants) as the baseline access model	Critical	Security
GOV-15	Keep ABAC UDFs simple, deterministic, and free of external calls	Required	Performance

ID	Rule	Severity	WAF Pillar
GOV-16	Grant BROWSE on catalogs for data discoverability	Required	Governance

GOV-01: Provision Identities via SCIM

Rule

All users, groups, and service principals must be provisioned at the Databricks **account level** via SCIM from your identity provider.

Why It Matters

- Ensures consistent identity management across all workspaces
- Enables unified access control for Unity Catalog objects
- Simplifies onboarding/offboarding through IdP automation

Implementation

```
Identity Provider (Okta, Azure AD, etc.)
    │
    └── SCIM Provisioning ──► Databricks Account
                                  │
                                  └── Automatic Sync to Workspaces
```

Common Mistakes

❌ Don't	✅ Do
Add users manually in Databricks	Provision through IdP with SCIM
Use workspace-level SCIM	Use account-level SCIM

GOV-02: Manage Groups in Identity Provider

Rule

All groups must be created and managed in your identity provider, then synced to Databricks via SCIM.

Why It Matters

- Single source of truth for organizational structure
- Changes propagate automatically across all Databricks resources
- Maintains consistency with enterprise identity governance

Recommended Groups

Group	Purpose

Group	Purpose
`data-platform-admins`	Infrastructure and account management
`data-engineers`	Bronze/Silver pipeline development
`analytics-engineers`	Gold layer and semantic development
`data-scientists`	ML and experimentation
`business-analysts`	Read-only access to Gold layer

Migration Note

Workspace-local groups do not sync automatically. Migrate them to account-level groups through your IdP.

GOV-03: Limit Admin Roles

Rule

Assign admin roles (`account admin`, `metastore admin`) sparingly. Avoid granting `ALL PRIVILEGES` or `MANAGE` permissions.

Why It Matters

- Reduces risk of accidental data exposure or deletion
- Follows least-privilege security principle
- Simplifies audit and compliance

Privilege Strategy

```
-- ✅ CORRECT: Specific grants
GRANT USE CATALOG ON CATALOG prod_catalog TO `data-engineers`;
GRANT SELECT ON SCHEMA prod_catalog.gold TO `business-analysts`;

-- ❌ WRONG: Overly broad
GRANT ALL PRIVILEGES ON CATALOG prod_catalog TO `data-engineers`;
```

Admin Role Limits

Role	Recommendation
Account Admin	2-3 people maximum
Metastore Admin	Only if required; optional
Catalog Owner	Team groups only

GOV-04: Catalog-Level Storage

Rule

Configure managed storage at the **catalog level** as the primary unit of data isolation.

Why It Matters

- Provides clear data isolation boundaries
- Simplifies storage security management
- Enables environment-specific storage policies

Implementation

```sql
-- Create catalog with dedicated storage
CREATE CATALOG production_catalog
MANAGED LOCATION 's3://prod-data/unity-catalog/';

-- Separate storage for development
CREATE CATALOG development_catalog
MANAGED LOCATION 's3://dev-data/unity-catalog/';
```

Storage Security

- Never reuse DBFS root bucket for managed storage
- Restrict IAM access to Databricks only
- Do not allow external service access to managed storage locations

---

GOV-05: Prefer Managed Tables

Rule

Use Unity Catalog managed tables for all new tables. External tables require documented justification.

Why It Matters

Feature	Managed	External
Auto compaction	✅	❌
Auto optimize	✅	❌
Full governance	✅	Limited
File lifecycle	Automatic	Manual

When External Tables Are Acceptable

- Migrating from Hive metastore (temporary)
- Disaster recovery requirements not met by managed
- External readers/writers required

GOV-06: Service Principals for Production

Rule

All production jobs must run as service principals, not individual users.

Why It Matters

- Eliminates dependency on individual user accounts
- Reduces accidental overwrites of production data
- Provides clear audit trail for automated processes
- Jobs continue running when employees leave

Implementation

```yaml
# In Asset Bundle
resources:
  jobs:
    production_etl:
      run_as:
        service_principal_name: "etl-service-principal"
```

GOV-07: Group Ownership

Rule

Assign ownership of catalogs, schemas, and tables to groups—never to individual users.

Why It Matters

- Ownership persists through employee turnover
- Enables team collaboration on data assets
- Scales with organizational growth

Implementation

```sql
-- Correct: Group ownership
ALTER CATALOG production SET OWNER TO `data-platform-admins`;
ALTER SCHEMA production.gold SET OWNER TO `analytics-engineers`;

-- Wrong: Individual ownership (avoid)
-- ALTER TABLE prod.gold.sales SET OWNER TO `john@company.com`;
```

GOV-08: Lineage-Driven Governance

Rule

Before making schema changes (adding, removing, or altering columns), use Unity Catalog lineage to assess downstream impact. All breaking changes require notification to affected consumers.

Why It Matters

- Prevents unplanned breakage of downstream dashboards, models, and pipelines
- Enables data product consumers to prepare for changes
- Supports data contract enforcement (EA-11)
- Creates an auditable change management trail

Implementation

```sql
-- Check downstream dependencies before altering a table
-- Use Unity Catalog lineage API or system tables
SELECT
    target_table_full_name,
    target_type,
    source_column_name
FROM system.access.table_lineage
WHERE source_table_full_name = 'catalog.gold.dim_customer'
ORDER BY target_table_full_name;
```

Change Management Process

Change Type	Impact Check	Notification
Add column	Low risk — verify no name conflicts	Informational
Rename column	High risk — check all downstream references	30-day notice + migration support
Drop column	High risk — identify all consumers	30-day deprecation, then remove
Change type	Medium risk — verify cast compatibility	14-day notice

---

GOV-09: Governance-as-Code

Rule

Automate governance enforcement using system tables, information schema queries, and scheduled validation jobs rather than relying solely on manual audits.

Why It Matters

- Scales governance across hundreds of tables and dozens of teams
- Provides continuous compliance rather than point-in-time audits
- Reduces human error in permission management
- Enables proactive alerting on governance violations

Implementation

Automated Compliance Checks (scheduled daily):

```sql
-- Check: All Gold tables have PRIMARY KEY constraints
SELECT
    t.table_catalog, t.table_schema, t.table_name,
    CASE WHEN tc.constraint_name IS NOT NULL THEN 'PASS' ELSE 'FAIL' END
AS pk_check
FROM information_schema.tables t
LEFT JOIN information_schema.table_constraints tc
    ON t.table_catalog = tc.table_catalog
    AND t.table_schema = tc.table_schema
    AND t.table_name = tc.table_name
    AND tc.constraint_type = 'PRIMARY KEY'
WHERE t.table_schema = 'gold';

-- Check: All tables have COMMENTs
SELECT
    table_catalog, table_schema, table_name,
    CASE WHEN comment IS NOT NULL AND LENGTH(comment) > 20 THEN 'PASS'
ELSE 'FAIL' END AS comment_check
FROM information_schema.tables
WHERE table_schema IN ('bronze', 'silver', 'gold');

-- Check: No ALL PRIVILEGES grants
SELECT
    grantee, privilege_type, table_catalog, table_schema
FROM information_schema.table_privileges
WHERE privilege_type = 'ALL PRIVILEGES';
```

Automation Patterns

Pattern	Implementation
Compliance dashboard	Scheduled job writing results to monitoring schema
Alerting	Databricks SQL Alert on compliance failures
Pre-merge validation	CI/CD check that bundle YAML passes governance rules
Drift detection	Compare actual state vs declared state in Git

---

GOV-10: Row Filters and Column Masks

Rule

Use row filters and column masks in Unity Catalog for fine-grained access control on sensitive data. Apply row filters to restrict which rows a user can see; apply column masks to redact or hash sensitive column values.

Why It Matters

- Enables sharing a single table across teams with different access levels
- Eliminates the need for separate filtered views per audience
- Enforces access control at the data layer, not the application layer
- Supports compliance requirements (GDPR, HIPAA) without data duplication

Implementation

```sql
-- Create a row filter function
CREATE FUNCTION gold.region_filter(region_param STRING)
RETURNS BOOLEAN
RETURN IF(IS_ACCOUNT_GROUP_MEMBER('global-analysts'), true, region_param =
current_user_region());

-- Apply row filter to a table
ALTER TABLE gold.fact_sales SET ROW FILTER gold.region_filter ON (region);

-- Create a column mask function
CREATE FUNCTION gold.mask_email(email_param STRING)
RETURNS STRING
RETURN IF(IS_ACCOUNT_GROUP_MEMBER('pii-readers'), email_param,
CONCAT('***@', SPLIT(email_param, '@')[1]));

-- Apply column mask
ALTER TABLE gold.dim_customer ALTER COLUMN email SET MASK gold.mask_email;
```

When to Use

Technique	Use Case
Row filter	Regional data access, tenant isolation, role-based row visibility
Column mask	PII redaction, salary hiding, partial email display
Both combined	Regulated datasets shared across multiple business units

Common Mistakes

Do Not	Instead
Create separate views per audience	Use row filters on a single table
Rely on application-level masking	Use UC column masks (enforced at query engine)
Apply masks without testing	Verify with `SELECT * AS <restricted_user>`

GOV-11: Configure Audit Logging

Rule

All workspaces must have audit logging enabled via system tables for compliance, security monitoring, and operational visibility.

Why It Matters

- Required for regulatory compliance (SOC 2, HIPAA, GDPR)
- Enables detection of unauthorized access attempts
- Provides evidence trail for security investigations
- Supports operational troubleshooting

Implementation

```sql
-- Query audit logs for recent access events
SELECT
    event_time,
    service_name,
    action_name,
    request_params,
    identity.email AS user_email
FROM system.access.audit
WHERE event_time >= current_date() - 7
ORDER BY event_time DESC;

-- Monitor for suspicious admin actions
SELECT *
FROM system.access.audit
WHERE action_name IN ('createCluster', 'deleteCluster',
'changePermissions')
  AND event_time >= current_date() - 1
ORDER BY event_time DESC;
```

GOV-12: Use ABAC Policies for Scalable Tag-Driven Access Control

Rule

Use Attribute-Based Access Control (ABAC) policies with governed tags for centralized, scalable fine-grained access control. Prefer ABAC over per-table row filters and column masks for new implementations.

Why It Matters

- ABAC policies defined once at catalog/schema level automatically apply to all child tables — no per-table configuration
- Governed tags drive dynamic enforcement: tag a table as `pii=true` and all applicable policies activate automatically
- Scales across hundreds of tables without individual privilege management
- Complements RBAC (privilege grants) with data-level filtering and masking

Access Control Layering

Layer	Mechanism	Purpose
1. Workspace bindings	Restrict catalogs to specific workspaces	Environment isolation
2. RBAC (privileges)	GRANT/REVOKE on securables	Baseline access control
3. ABAC (policies)	Tag-driven row filters and column masks	Dynamic fine-grained access
4. Table-level filters	Per-table UDFs	Legacy or table-specific logic

Implementation

Row filter policy (hide EU data from US analysts):

```
-- Step 1: Create a row filter UDF
CREATE FUNCTION governance.filter_by_region(region STRING, allowed
ARRAY<STRING>)
RETURNS BOOLEAN
DETERMINISTIC
RETURN array_contains(TRANSFORM(allowed, x -> lower(x)), lower(region));

-- Step 2: Create ABAC policy on catalog
CREATE POLICY hide_eu_customers
ON CATALOG prod_catalog
COMMENT 'Hide EU customer rows from US analysts'
ROW FILTER governance.filter_by_region
TO `us_analysts`
FOR TABLES
MATCH COLUMNS
    hasTagValue('geo_region', 'EMEA') AS region
USING COLUMNS (region);
```

Column mask policy (mask PII):

```
-- Step 1: Create a masking UDF
CREATE FUNCTION governance.mask_ssn(ssn STRING)
RETURNS STRING
DETERMINISTIC
RETURN CASE
    WHEN ssn IS NULL THEN NULL
    WHEN LENGTH(ssn) >= 4 THEN CONCAT('XXX-XX-', RIGHT(REGEXP_REPLACE(ssn,
'[^0-9]', ''), 4))
    ELSE 'MASKED'
END;

-- Step 2: Create ABAC policy
CREATE POLICY mask_ssn_policy
ON CATALOG prod_catalog
COMMENT 'Mask SSN columns for non-compliance users'
COLUMN MASK governance.mask_ssn
TO `all_users`
```

```
EXCEPT `compliance_team`
FOR TABLES
MATCH COLUMNS
    hasTagValue('pii', 'ssn') AS ssn
ON COLUMN ssn;
```

When to Use ABAC vs Per-Table Filters

Scenario	Use ABAC	Use Per-Table Filters
PII masking across catalog	Yes	No — doesn't scale
Regional data partitioning	Yes	No
Single table with unique logic	No	Yes
Delta Sharing recipients	No — not supported	Use dynamic views
Materialized views	Only if pipeline owner is exempt	Use views

Limitations

- Requires DBR 16.4+ or serverless compute
- Cannot apply ABAC directly to views (view owner's permissions are used)
- Only one distinct row filter per table per user at runtime
- Only one distinct column mask per column per user at runtime
- Max 3 column conditions in `MATCH COLUMNS` clause
- Policy quotas: 10 per catalog, 10 per schema, 5 per table

GOV-13: Define ABAC Policies at Catalog Level

Rule

Define ABAC policies at the highest applicable level (catalog preferred) to maximize governance efficiency through automatic inheritance to child schemas and tables.

Why It Matters

- A single catalog-level policy governs all tables in all schemas — no per-table configuration
- Reduces administrative overhead and policy duplication
- New tables automatically inherit policies when tagged with governed tags
- Consistent enforcement across the data hierarchy

Policy Hierarchy

```
Catalog Policy (broadest)
    └── Schema Policy (mid-level)
         └── Table Policy (narrowest)
```

- Catalog policies automatically apply to all schemas and tables within
- Schema policies apply to all tables within
- Table policies are the most granular

Implementation

```
-- Define a catalog-level row filter policy
CREATE POLICY regional_access
ON CATALOG prod_catalog
COMMENT 'Enforce regional data access across all schemas'
ROW FILTER governance.filter_by_region
TO `regional_users`
FOR TABLES
MATCH COLUMNS
    hasTag('geo_region') AS region
USING COLUMNS (region);
```

Anti-Patterns

- Creating identical policies on every table instead of defining once at catalog level
- Mixing ABAC policies with per-table row filters on the same table (causes conflicts)
- Defining overlapping policies that result in multiple distinct filters for the same user

---

GOV-14: Use RBAC as the Baseline Access Model

Rule

Use Role-Based Access Control (RBAC) through privilege grants as the foundation of all data access. Layer ABAC on top for fine-grained, dynamic data-level controls.

Why It Matters

- RBAC (GRANT/REVOKE) controls WHO can access WHAT objects
- ABAC controls WHAT DATA users see within those objects
- Without RBAC baseline, ABAC policies have no effect (users need SELECT privilege first)
- Follows least-privilege: only grant what each role needs

Privilege Assignment Strategy

```
-- Step 1: RBAC — Grant object-level access
GRANT USE CATALOG ON CATALOG prod_catalog TO `data-engineers`;
GRANT USE SCHEMA ON SCHEMA prod_catalog.gold TO `business-analysts`;
GRANT SELECT ON SCHEMA prod_catalog.gold TO `business-analysts`;

-- Step 2: ABAC — Layer data-level controls
-- (ABAC policies filter/mask data within the tables they can already
access)
```

Privilege Hierarchy

Privilege	Scope	Inherits To
USE CATALOG	Catalog	All schemas
USE SCHEMA	Schema	All tables/views
SELECT	Table/View	—
MODIFY	Table	—
CREATE TABLE	Schema	—
BROWSE	Catalog	All child objects (metadata only)

Key Rules

- Never grant `ALL PRIVILEGES` — always use specific grants (see GOV-03)
- Use groups for grants, not individual users (see GOV-07)
- Object ownership provides full control — assign to groups, not individuals
- `MANAGE` privilege allows granting access without ownership transfer

Anti-Patterns

```
-- WRONG: Granting ALL PRIVILEGES
GRANT ALL PRIVILEGES ON CATALOG prod_catalog TO `data-team`;

-- WRONG: Granting to individual users
GRANT SELECT ON TABLE prod.gold.customers TO `john@company.com`;

-- CORRECT: Specific grants to groups
GRANT SELECT ON SCHEMA prod_catalog.gold TO `business-analysts`;
```

GOV-15: Keep ABAC UDFs Simple and Deterministic

Rule

ABAC UDFs (row filters and column masks) must be simple SQL functions that are deterministic, avoid external calls, and reference only built-in functions and target table columns.

Why It Matters

- ABAC UDFs execute on EVERY query for EVERY applicable row — inefficient UDFs become governance bottlenecks
- Non-deterministic UDFs break caching, vectorization, and consistent joins
- External API calls in UDFs cause timeouts, single points of failure, and blocked parallel execution
- Simple SQL UDFs enable predicate pushdown and query optimization

UDF Best Practices

Do	Don't
Use simple `CASE` statements	Call external APIs or databases
Stay deterministic (same input = same output)	Use `is_member()` or `is_account_group_member()` inside UDFs
Use only built-in SQL functions	Call other UDFs from within a UDF
Reference only target table columns	Use dynamic SQL generation
Mark as `DETERMINISTIC`	Apply heavy regex on large text fields
Test at scale (1M+ rows)	Use metadata lookups per row

Keep Access Checks in Policies, Not UDFs

```
-- CORRECT: UDF focuses on data transformation only
CREATE FUNCTION governance.mask_email(email STRING)
RETURNS STRING
DETERMINISTIC
RETURN CONCAT('***@', SPLIT(email, '@')[1]);

-- ABAC policy defines WHO and WHEN
CREATE POLICY mask_emails
ON CATALOG prod_catalog
COLUMN MASK governance.mask_email
TO `general_users`
EXCEPT `pii_readers`
FOR TABLES
MATCH COLUMNS hasTagValue('pii', 'email') AS email_col
ON COLUMN email_col;
```

```
-- WRONG: Access check inside UDF
CREATE FUNCTION governance.mask_email_wrong(email STRING)
RETURNS STRING
RETURN IF(is_account_group_member('pii-readers'), email, CONCAT('***@',
SPLIT(email, '@')[1]));
-- Problem: is_account_group_member() is slow, per-row metadata lookup
```

Performance Testing

```
-- Validate UDF performance at scale before production
WITH test_data AS (
    SELECT CONCAT('user', seq, '@company.com') AS email
    FROM range(1000000)
)
```

```sql
SELECT COUNT(*), governance.mask_email(email) AS masked
FROM test_data
GROUP BY masked;
```

GOV-16: Grant BROWSE on Catalogs for Data Discoverability

Rule

Grant the `BROWSE` privilege on all catalogs to the `all-account-users` group to enable data discovery without granting data access.

Why It Matters

- Users can discover available data assets without requiring SELECT access
- Enables self-service access request workflows
- Supports Unity Catalog search and Genie data exploration
- Users who discover objects they need can submit access requests

Implementation

```sql
-- Enable metadata browsing for all users
GRANT BROWSE ON CATALOG prod_catalog TO `all-account-users`;
GRANT BROWSE ON CATALOG dev_catalog TO `all-account-users`;
```

What BROWSE Allows

Action	Allowed
See object names and metadata	Yes
View table/column COMMENTs	Yes
View tags and governed tags	Yes
Read data (SELECT)	No — requires separate SELECT grant
Request access	Yes — via access request workflow

Access Request Configuration

Configure access request destinations (email, Slack, Teams, webhook) so that users who discover objects can request access from the appropriate data steward.

Privilege Assignment Quick Reference

Required Privileges by Role

Role	USE CATALOG	USE SCHEMA	SELECT	MODIFY
Platform Admins	All	All	All	All
Data Engineers	All	Bronze/Silver	Bronze/Silver	Bronze/Silver
Analytics Engineers	All	Silver/Gold	Silver/Gold	Gold
Business Analysts	All	Gold only	Gold only	None

Enable Data Discovery

```
-- Allow all users to browse catalog metadata
GRANT BROWSE ON CATALOG production TO `all-account-users`;
```

Validation Checklist

Identity Management

- [ ] SCIM provisioning configured at account level
- [ ] Groups defined and managed in IdP
- [ ] No workspace-local groups in use

Privileges

- [ ] Service principals used for production jobs
- [ ] Ownership assigned to groups
- [ ] No ALL PRIVILEGES grants
- [ ] BROWSE granted for discoverability

Storage

- [ ] Catalog-level managed storage configured
- [ ] Managed tables used (external tables documented)
- [ ] Storage buckets not accessible outside Unity Catalog

Governance Automation

- [ ] Lineage checked before schema changes
- [ ] Automated compliance checks scheduled
- [ ] Governance dashboard operational
- [ ] Breaking change notifications configured

ABAC & Access Control

- [ ] ABAC policies defined at catalog level where possible (GOV-12, GOV-13)
- [ ] RBAC privileges granted as baseline before ABAC layering (GOV-14)
- [ ] ABAC UDFs are simple, deterministic SQL functions (GOV-15)

- ☐ No `is_member()` or external calls inside UDFs (GOV-15)
- ☐ BROWSE granted on all catalogs for discoverability (GOV-16)
- ☐ Access request destinations configured (GOV-16)

References

- Unity Catalog Best Practices
- Manage Users and Groups
- Manage Privileges
- ABAC Overview
- ABAC Policies
- UDFs for ABAC Best Practices
- Access Control in Unity Catalog